Serial Python Minimalis: Tutorial #07-Masukan/Keluaran Berkas (*File IO*)

Ahmad R. T. Nugraha

ver. 21 April 2026

Sering kali kita perlu membaca data ke dalam program Python, atau menuliskan data yang dihasilkannya ke sebuah berkas (*file*). Untuk keperluan ini, kode Python dapat berinteraksi dengan file data eksternal melalui sebuah objek file (*file object*), yang dibuat menggunakan fungsi bawaan `open`.

1 Menulis File

Untuk membuat objek file yang akan ditulis, kita dapat memberikan fungsi `open` nama filenya, dan menetapkan `'w'` (*write*) sebagai modenya. Di sini, kita akan menetapkan variabel `f` ke objek file yang diberikan:

```
f = open('output_file.txt', 'w')
```

Output dari fungsi `print` yang biasanya dikirim ke layar kemudian dapat dialihkan ke objek file yang terbuka ini dengan menambahkan argumen `file=f`:

```
print('a line of data', file=f)
```

Setelah selesai menuliskan semua output, objek file harus ditutup dengan memanggil fungsi `close`-nya:

```
f.close()
```

Sekarang, jika kita periksa file teks `output_file.txt`, akan didapati bahwa file tersebut berisi satu baris bertuliskan `a line of data`.

Pada praktiknya dalam Python, lebih disarankan untuk menggunakan sintaksis yang sedikit berbeda untuk memasukkan dan mengeluarkan data ke sebuah file:

```
with open('output_file.txt', 'w') as f:
    print('a line of data', file=f)
    print('a second line of data', file=f)
```

Kata kunci `with` menciptakan sebuah konteks (*context*) agar objek file `f` dapat digunakan. Konteks ini memastikan bahwa file tersebut secara otomatis ditutup setelah blok kodenya (yang diindentasi) selesai dieksekusi sehingga tidak perlu lagi memanggil `f.close()`. Selain itu, semua kesalahan apa pun yang mungkin terjadi selama proses penulisan file dapat ditangani guna meminimalkan risiko kehilangan data.

Untuk menulis isi yang sama persis dari sebuah string ke file (tanpa tambahan karakter baris baru (*newline*) atau sejenisnya), kita panggil metode `f.write`:

```
with open("data.txt", "w") as f:
    f.write("text1")
    f.write("text2")
```

Hasilnya adalah file teks dengan isi `text1text2` (tanpa karakter baris baru atau spasi apa pun).

2 Membaca File

Untuk membaca file data teks, buka dengan mode `'r'` (*read*). Mode ini adalah bawaan (*default*) fungsi `open` sehingga sebenarnya tidak perlu dituliskan secara eksplisit.

Ada beberapa metode objek file untuk membaca dari file yang sedang terbuka:

- `f.read()` (untuk membaca keseluruhan file),
- `f.readline()` (membaca satu baris tunggal dari file),
- atau `f.readlines()` (membaca semua baris dan menjadikannya sebuah `list` berisi string).

```
with open('output_file.txt') as f:
    data = f.read()

data
```

Output:

```
'a line of data\na second line of data\n'
```

Perhatikan adanya karakter penanda akhir baris (*line-ending*) yang di-*escape*, yaitu `\n`. Untuk melihat string data tersebut sebagaimana tampilannya di penyunting teks (*text editor*), kita dapat mencetaknya:

```
print(data)
```

Output:

```
a line of data
a second line of data
```

Sebagai alternatif:

```python
with open('output_file.txt') as f:
    data = f.readlines()

data
```

Output:

```
['a line of data\n', 'a second line of data\n']
```

Di sini, `data` adalah sebuah `list` Python yang berisi dua baris dari file tersebut (termasuk karakter akhir barisnya).

Tabel 1: Mode buka file pada Python

Karakter	Deskripsi
`'r'`	buka untuk membaca (*default*)
`'w'`	buka untuk menulis, menimpa file jika sudah ada
`'x'`	buka untuk menulis, akan gagal jika file sudah ada
`'a'`	buka untuk menulis, menambahkan ke akhir file (*append*) jika file sudah ada
`'t'`	baca/tulis data teks (*default*)
`'b'`	baca/tulis data biner
`'+'`	buka untuk pembaruan (*updating* – membaca dan menulis)

Python juga dapat membaca dan menulis file biner, serta menambahkan isi baru ke file yang sudah ada dengan menyertakan tanda (*flag*) tambahan ke dalam argumen mode (lihat tabel 1). Misalnya, `open('file.dat', 'xb')` membuka file bernama `'file.dat'` untuk menulis data biner, tetapi akan gagal dieksekusi jika file tersebut ternyata sudah ada sebelumnya.

3 Pengodean Karakter (Character Encoding)

Komputer merepresentasikan karakter teks yang dapat dibaca manusia sebagai sekumpulan angka. Pemetaan yang mengaitkan setiap angka (titik kode / *code point*) ke karakter tertentu disebut dengan pengodean karakter (*character encoding*).

Banyak komputer di era awal menggunakan pengodean ASCII (*American Standard Code for Information Interchange*) yang merepresentasikan himpunan 95 karakter sebagai angka antara 32 dan 126 (misalnya, 65 = "A"). Sayangnya, hingga saat ini, belum ada standar yang disepakati secara luas untuk pengodean perluasan (*expanded encoding*) yang diperlukan untuk merepresentasikan ribuan karakter yang digunakan dalam ratusan bahasa manusia yang berbeda-beda. Selain itu, sistem operasi dan platform yang berbeda sering kali menggunakan sistem pengodean yang berbeda pula.

Jika kita pernah melihat teks yang ditampilkan berantakan seperti "caf¿" atau "cafÃ©" padahal seharusnya "café", penyebabnya kemungkinan besar adalah ketidakcocokan antara pengodean string yang diasumsikan oleh program dan yang sebenarnya digunakan oleh si penulis aslinya.

Pengodean yang digunakan oleh Python dalam membaca dan menulis file teks bergantung pada platform sistem operasi. File yang ditulis serta dibaca pada platform yang sama seharusnya tidak akan mengalami perubahan apa-apa. Namun, terkadang kita perlu menentukan pengodean secara eksplisit saat membuka suatu file, terutama jika file tersebut berasal dari tempat atau sistem operasi lain. Pengodean secara eksplisit dapat dilakukan menggunakan argumen `encoding`. Karena pengodean karakter yang paling banyak diadopsi penggunaannya di dunia saat ini adalah UTF-8, kita bisa mengadopsinya sebagai titik awal:

```
with open("data.txt", encoding="utf8") as f:
    print(f.read())
```

Pengecualian utama dari tren ini adalah (tentu saja) Microsoft Windows, yang secara historis lebih menyukai pengodean `cp1252`. Sebenarnya terdapat sedikit irisan (*overlap*) di antara sistem-sistem pengodean karakter (misalnya, 95 karakter cetak pertama yang dikodekan oleh UTF8 sama persis dengan yang ada di set karakter ASCII), tetapi jika proses membaca file gagal atau memberikan hasil yang kacau karena alasan tertentu, kita perlu mendeduksi atau menebak pengodeannya dengan cara tertentu.

Studi Kasus

Kasus 1 Menulis dan Membaca File Teks

Tuliskan sebuah file teks bernama `atomic_numbers.txt` yang berisi daftar simbol dari sepuluh unsur pertama pada tabel periodik beserta nomor atomnya, Z.

Solusi

Pertama-tama, buatlah sebuah `list` terurut yang berisi simbol-simbol unsur tersebut. Nomor atom dari setiap unsur adalah indeksnya di dalam `list` ditambah satu (karena indeks pada Python dimulai dari nol).

```
elements = 'H He Li Be B C N O F Ne'
elements = elements.split()
elements
```

Output:

```
['H', 'He', 'Li', 'Be', 'B', 'C', 'N', 'O', 'F', 'Ne']
```

Untuk menulis ke dalam file teks, buatlah sebuah objek file, `fo`, dalam mode tulis (`'w'`) lalu iterasi `list` elemen tersebut. Kita dapat menggunakan fungsi `enumerate` untuk mendapatkan nilai indeks dan simbol unsur secara bersamaan pada setiap iterasinya:

```
with open('atomic_numbers.txt', 'w') as fo:
    for i, symbol in enumerate(elements):
        print(f'{i+1:2d} {symbol}', file=fo)
```

Untuk membaca file teks tersebut, kita dapat membuat objek file yang lain, kali ini dalam mode baca (`'r'`) dan membaca seluruh barisnya menjadi sebuah `list` yang berisi sekumpulan string:

```
with open('atomic_numbers.txt', 'r') as fi:
    lines = fi.readlines()

lines
```

Output:

```
[' 1 H\n', ' 2 He\n', ' 3 Li\n', ' 4 Be\n', ' 5 B\n', ' 6 C\n', ' 7
↪  N\n', ' 8 O\n', ' 9 F\n', '10 Ne\n']
```

Setiap baris di atas mencakup karakter baris baru (*newline*), yaitu `'\n'`, tepat di bagian akhirnya. Untuk membuat outputnya terlihat lebih rapi, kita bisa mengiterasi `list` ini dan mengurai (*parse*) kolom-kolomnya secara terpisah. Terdapat dua kolom pada setiap baris yang dipisahkan oleh spasi kosong (*whitespace*):

```
with open('atomic_numbers.txt', 'r') as fi:
    for line in fi.readlines():
        fields = line.split()
        Z = int(fields[0])
        symbol = fields[1]
        print(f'Z({symbol:2s}) = {Z:2d}')
```

Output:

```
Z(H ) =  1
Z(He) =  2
Z(Li) =  3
Z(Be) =  4
Z(B ) =  5
Z(C ) =  6
Z(N ) =  7
Z(O ) =  8
Z(F ) =  9
Z(Ne) = 10
```

Kasus 2 Termodinamika Reaksi

File `thermo-data.csv` (yang dapat diunduh dari folder `dat` pada repositori `https://github.com.com/BRIN-Q/Python-minimalis`), berisi data termodinamika untuk beberapa senyawa pada kondisi tekanan standar ($p = 1$ bar) dan pada $T = 298$ K dengan format:

```
Formula, DfHm/kJ.mol-1, Sm/J.K-1.mol-1
H2O(g), -241.83, 188.84
CO2 (g), -393.52, 213.79
LiOH(s), -484.93, 42.81
Li2CO3(s), -1216.04, 90.31
```

Bacalah data ini ke dalam *dictionary* yang sesuai untuk entalpi pembentukan molar standar dan entropi molar standar. Gunakan data tersebut untuk menghitung konstanta kesetimbangan dari reaksi berikut, yang menjadi dasar operasi pembuangan karbon dioksida dari kabin awak pada beberapa wahana antariksa:

$$2\mathrm{LiOH}(s) + \mathrm{CO_2}(g) \rightarrow \mathrm{Li_2CO_3}(s) + \mathrm{H_2O}(g)$$

Solusi

Konstanta kesetimbangan (K) dapat dihitung dari perubahan energi bebas Gibbs standar ($\Delta_r G^\ominus$), dengan $\Delta_r G^\ominus = \Delta_r H^\ominus - T\Delta_r S^\ominus$.

```python
import math

DfH = {}
S = {}

with open('dat/thermo-data.csv', 'r') as f:
    header = f.readline()
    for line in f:
        fields = line.split(',')
        formula = fields[0].strip() # Menghapus spasi ekstra jika ada
        DfH[formula] = float(fields[1])
        S[formula] = float(fields[2])

# Reaksi: 2 LiOH(s) + CO2(g) -> Li2CO3(s) + H2O(g)
# Stoikiometri (produk bernilai positif, reaktan bernilai negatif)
nu = {'LiOH(s)': -2, 'CO2 (g)': -1, 'Li2CO3(s)': 1, 'H2O(g)': 1}

# Hitung Delta H dan Delta S reaksi
DrH = sum(nu[comp] * DfH[comp] for comp in nu) # dalam kJ/mol
DrS = sum(nu[comp] * S[comp] for comp in nu)   # dalam J/(K.mol)

# Hitung Delta G reaksi dan Konstanta Kesetimbangan
T = 298.15
DrG = DrH * 1000 - T * DrS # konversi DrH ke J/mol
R = 8.31446

K = math.exp(-DrG / (R * T))

print(f"Entalpi Reaksi (DrH)   = {DrH:.2f} kJ/mol")
print(f"Entropi Reaksi (DrS)   = {DrS:.2f} J/(K.mol)")
print(f"Energi Gibbs (DrG)     = {DrG/1000:.2f} kJ/mol")
print(f"Konstanta Kesetimbangan= {K:.2e}")
```

Output:

```
Entalpi Reaksi (DrH)   = -94.49 kJ/mol
Entropi Reaksi (DrS)   = -20.26 J/(K.mol)
Energi Gibbs (DrG)     = -88.45 kJ/mol
Konstanta Kesetimbangan= 3.12e+15
```

Reaksi ini sangat disukai secara termodinamika (spontan) ke arah produk, dibuktikan dengan nilai tetapan kesetimbangan K yang sangat besar ($\sim 10^{15}$).